Week 3: Python Programming II

Objects, Errors, Vectorization

Theory: What You Need This Week

This week is focused on transitioning your programming skills from basic scripts to robust, production-ready software systems. The curriculum ties together three core components into a single system:

- **Object-Oriented Programming (OOP):** encapsulate state and behavior in classes.
- **Deliberate error handling:** ensure your code survives noisy, real-world data without crashing.
- **Computational performance:** make your operations orders of magnitude faster using NumPy (vectorization and broadcasting).

1. Topics at a Glance

- **Classes and instances:** `class`, `__init__`, `self`, instance attributes, methods.
- **Encapsulation:** public, protected (`_name`), and private (`__name`) members; controlled access through methods.
- **Dunder methods:** `__repr__`, `__str__`, container protocols, and how objects integrate with built-in Python.
- **Class-level features:** `@classmethod` as an alternative constructor; `@property` for computed attributes.
- **Inheritance and `super()`:** extending behavior without copying code; method overriding and polymorphism.
- **Error handling:** `raise`, `try` / `except` / `else` / `finally`, catching specific types, and defining custom exception classes.
- **NumPy fundamentals:** `ndarray`, `dtype`, axis-aware reductions, boolean masking.
- **Vectorization and broadcasting:** elementwise operations on whole arrays; honest benchmarking with warm-up and best-of-N timing.

2. Learning Pathway

Content

This week is focused on **Python Programming II (Objects, Errors, Vectorization)**, the engineering layer that turns working scripts into reliable software. It provides the paradigms of object-oriented design (classes, inheritance, polymorphism), the discipline of deliberate error handling (raising and catching specific exceptions), and the computational tools (NumPy vectorization, broadcasting) required to write code that is clean, robust, and fast enough to scale beyond toy examples.

This knowledge is useful for courses like 04-645 Internet of Things (IoT) or 18-731 Network Security, where assignments frequently require sensor and device abstractions defined as Python classes, custom exceptions for malformed packet payloads in stream parsing, and vectorized signal-processing operations over arrays of telemetry samples to detect anomalies in real time.

How to use this pathway

- Follow the table from top to bottom.
- The main resources are video tutorials from LinkedIn Learning. CS50P notes are the free institutional alternative.
- If you already know OOP and just need a refresher, jump directly to the hands-on row of each topic.
- Start at your tier: **Foundation** from Topic 1; **Core** from Topic 3; **Extension** from Topic 5.
- **Estimated total time:** up to 4 hours.

Resources

- **Concept videos (primary):** *Programming Foundations: Object-Oriented Design* and *Python Object-Oriented Programming* (LinkedIn Learning).
- **Free institutional notes:** Harvard CS50P Week 8 (OOP) and Week 3 (Exceptions).
- **NumPy primer:** Stanford CS231n NumPy Tutorial and *NumPy: the absolute basics for beginners*.
- **Depth resource:** *From Python to NumPy* by Nicolas Rougier (INRIA, open access).

Weekly Topics

Topic	Focus and resources	Time (mins)	Tier
0	**Setup and environment** Confirm Python 3.10+ and `pip install numpy`.	5	All
1	**OOP from scratch** *Concept:* class definitions, instantiation, attributes, methods. *Resources:* *OOP Foundations* | CS50P Week 8	25 + 10	Foundation
2	**Pythonic OOP** *Concept:* `__init__`, `self`, `__repr__`, instance/class vars. *Resources:* *Python OOP* | Python Tutorial Ch. 9	30 + 5	Foundation
3	**Inheritance & properties** *Concept:* `super()`, `@classmethod`, `@property`. *Resources:* *Python OOP*.	25 + 15	Core
4	**Error handling** *Concept:* `try` / `except` / `else` / `finally`, custom exceptions. *Resources:* CS50P Week 3 | Python Tutorial Ch. 8	35 + 15	Core
5	**NumPy arrays** *Concept:* dtypes, axis, boolean masks. *Resources:* CS231n NumPy | NumPy Basics	25 + 10	Extension
6	**Vectorization & broadcasting** *Concept:* array operations, align dimensions, benchmarking. *Resources:* *Python to NumPy*.	25 + 15	Extension

3 Concept Map

This table maps each architectural concept to its engineering purpose and to where it is exercised in your weekly mastery challenge.

Concept	Why it matters	Where exercised
Classes, `__init__`, methods	Encapsulates state and behavior together; replaces loose, parallel lists and fragmented functions.	Mission Level 1
`__repr__` and dunder methods	Ensures objects print readably for developers and behave like standard Python containers.	Mission Levels 1 and 2
`@classmethod`, `@property`	Enables alternative constructors (e.g. loading from files) and clean computed attributes.	Mission Level 2
Inheritance, `super()`	Extends functional behavior without duplicating source code.	Mission Levels 2 and 3
`try` / `except` / `else` / `finally`	Allows code to survive bad or corrupt formatting in real-world data feeds.	Mission Level 3
Custom exceptions	Creates descriptive errors that carry critical diagnostic metadata (e.g. error coordinates).	Mission Level 3
Vectorization and broadcasting	Achieves orders-of-magnitude speed-ups on large mathematical matrices.	Mission Level 4
Honest benchmarking	Prevents measurement errors from cold-starts or caching; produces reproducible speed claims.	Mission Level 4

4 Rationale and Progression

Rationale. Last week you wrote scripts. This week you write *software*. The same logic is wrapped in classes that own their state, defended by exceptions that fail loudly on bad input, and accelerated by NumPy so it scales to data sizes that matter. These three habits compound: a class without error handling is fragile; error handling without vectorization is slow at scale; vectorization without classes is unmaintainable.

The seven topics are sequenced so each one earns the next. You cannot meaningfully write `__repr__` (Topic 2) before defining your first class (Topic 1); inheritance (Topic 3) is meaningless without classes; custom exceptions (Topic 4) become useful only once your classes have invariants worth defending; NumPy (Topics 5 and 6) becomes worth the cognitive cost only once your data structures are stable enough to operate on in bulk.

Completing each topic demonstrates a specific competency:

Topic	What it demonstrates
Topic 1	**Modeling state and behavior:** you can group attributes and the operations on them into one named, reusable unit.
Topic 2	**Pythonic object design:** your objects integrate with Python's built-in protocols (printing, equality, container access) instead of fighting them.
Topic 3	**Extending without copying:** you can add new behavior to an existing hierarchy through inheritance and overriding, with one source of truth per concept.
Topic 4	**Defensive engineering:** you can distinguish recoverable from unrecoverable failure, raise the right exception type at the right boundary, and catch it where the right response lives.
Topic 5	**Bulk-data thinking:** you can stop iterating over individual elements and start operating on whole arrays, with dtypes and axes as first-class design choices.
Topic 6	**Performance literacy:** you can rewrite a hot loop as a vectorized expression, prove the speed-up with an honest benchmark, and explain where the gain comes from.